



Sistemas Embebidos I

Otoño 2026

Práctica 03: Timing & SysTick

Camila Rodríguez Rosas

Ana Karen Rojas Ledesma

Jonathan Hernández Lazcano

Sábado 29 de agosto del 2026

Índice

<i>Práctica 3: Timing & Systick</i>	2
Introducción	2
Descripción de la práctica.....	2
Objetivo.....	2
Marco teórico.....	2
Desarrollo	4
Materiales	4
Esquemático	5
Desarrollo	6
Resultados	7
Evidencias	8
Conclusiones	9
Declaración de IA	10
Referencias	10

Práctica 3: Timing & Systick

Introducción

Descripción de la práctica

Implementar un temporizador de sistema con IRQ y trazas por toggling, y medir jitter.

Objetivo

Implementar un temporizador de sistema en una Raspberry Pi Pico 2W mediante el periférico TIMER y una interrupción por hardware (IRQ), de modo que la ALARM0 genere periódicamente una interrupción cada 250 ms. En cada interrupción, el sistema deberá cambiar el estado del LED conectado al GPIO15 y generar una señal de toggling en el GPIO14 para su observación mediante un osciloscopio. El funcionamiento del temporizador se verificará midiendo el periodo de la señal en GPIO14 y calculando el jitter mediante la diferencia entre el periodo máximo y mínimo, así como su desviación estándar.

Marco teórico

Temporizador del sistema

El microcontrolador RP2350 incluye dos bloques de temporizador del sistema (TIMER0 y TIMER1) independientes, los cuales funcionan con contadores de 64 bits de libre ejecución que se incrementan una vez cada microsegundo. El RP2350 permite utilizar un contador para generar eventos en instantes determinados. El valor del contador puede consultarse y utilizarse para establecer una alarma, de manera que cuando el contador alcanza el valor programado se genera el evento correspondiente.

En la práctica se utiliza el temporizador para generar una interrupción periódica. El intervalo seleccionado es de 250 ms. Debido a que el temporizador trabaja con una base de tiempo expresada en microsegundos, el intervalo debe convertirse antes de utilizarse en el programa.

Alarmas del temporizador

Una alarma es un mecanismo del temporizador que permite establecer un instante específico en el que debe producirse un evento. En el código se utiliza la **ALARM0**, identificada mediante ALARM_NUM 0. La primera alarma se programa tomando como referencia el valor actual del contador y sumándole el intervalo deseado. Posteriormente, cada vez que se atiende la interrupción, se calcula un nuevo instante sumando nuevamente 250ms al valor anterior.

Interrupciones por Hardware (IRQ)

Una IRQ (*Interrupt Request*) es una solicitud de interrupción generada por un periférico para indicar al procesador que ocurrió un evento que requiere atención. Cuando una interrupción está habilitada, el procesador puede suspender temporalmente la ejecución normal del programa y ejecutar algo asociado a la interrupción.

En esta práctica, la ALARM0 del temporizador está asociada con TIMER0_IRQ_0. La función `on_alarm_irq()` actúa como rutina de atención de la interrupción. Dentro de esta rutina se limpia la bandera de interrupción, se modifica el estado de las salidas GPIO y se programa el siguiente evento del temporizador. Su uso permite que el temporizador genere los eventos sin necesidad de esperar los 250ms.

Toggling

El *toggling* consiste en invertir periódicamente el estado lógico de una salida digital. En este caso, GPIO14 comienza en LOW y cambia de estado cada vez que se atiende la ALARM0. Debido a que el pin cambia de estado en cada interrupción, una transición ocurre cada 250 ms aproximadamente.

Jitter

El *jitter* representa la variación temporal de un evento respecto al comportamiento periódico esperado. En un temporizador ideal, los intervalos entre eventos serían idénticos. En un sistema real pueden existir pequeñas variaciones debido al funcionamiento del procesador, la atención de las interrupciones y otros factores relacionados con la ejecución del sistema.

A partir de mediciones en el osciloscopio, el jitter puede identificarse mediante la diferencia entre el valor máximo y mínimo. La medición experimental permite determinar cuánto se desvía el comportamiento real del sistema respecto al intervalo programado de 250 ms.

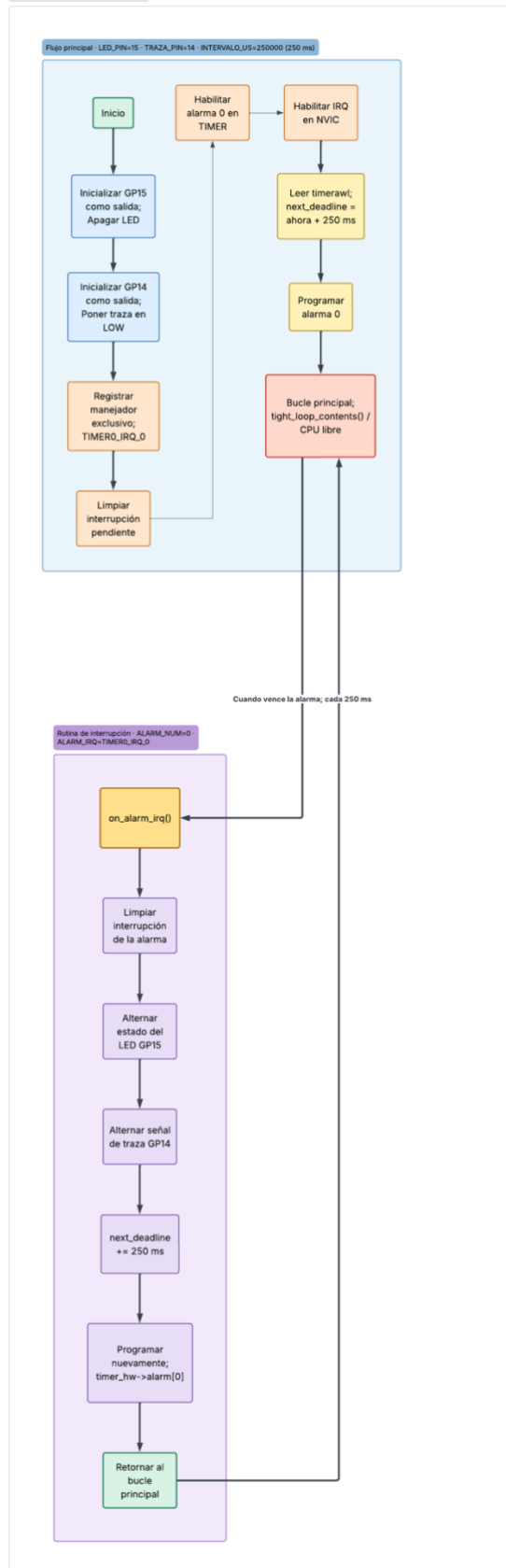
Desarrollo

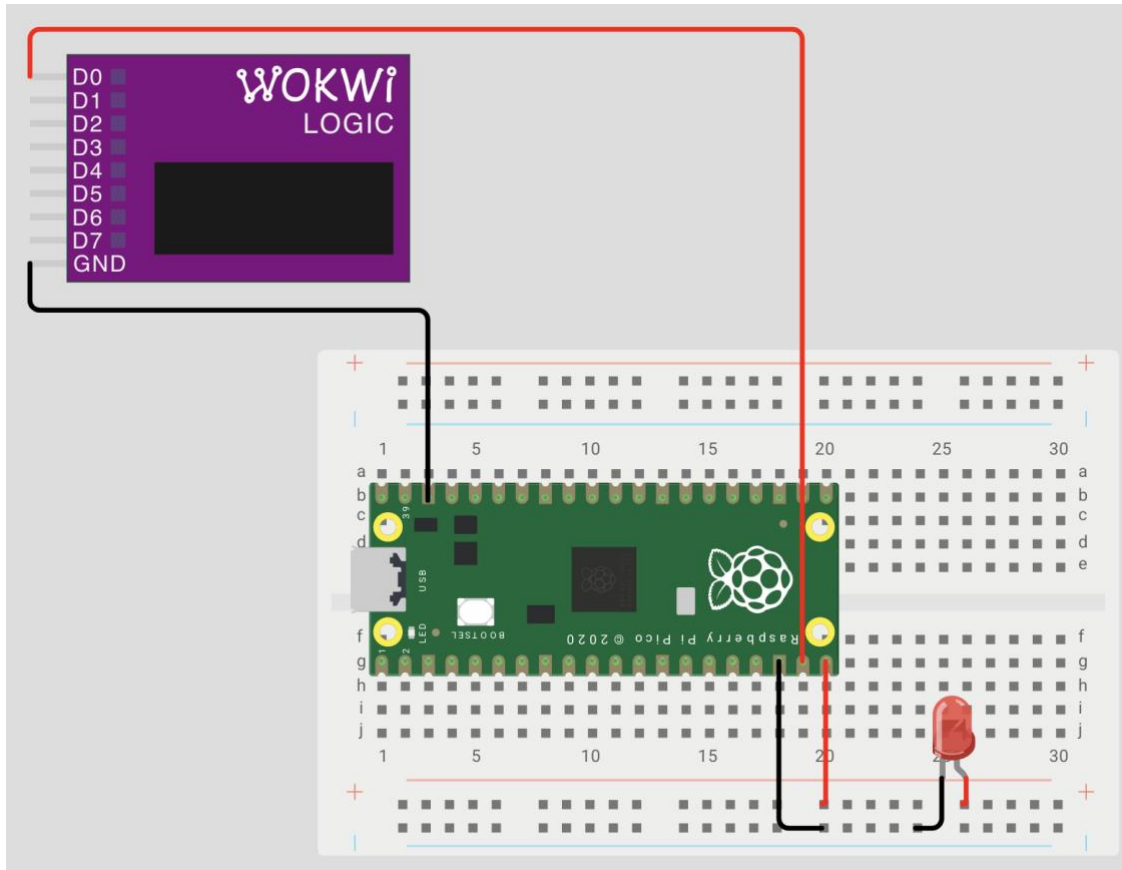
Materiales

- Raspberry Pi Pico 2W
- LED rojo
- Protoboard
- Cable micro USB
- Jumpers
- Osciloscopio
- Punta para osciloscopio

Software: Visual Studio Code

RP2040: Temporizador e Interrupciones





Desarrollo

```

1  #include
2  #include "pico/stdlib.h"
3  #include "hardware/timer.h"
4  #include "hardware/irq.h"
5  #include "hardware/structs/sio.h"
6
7  #define LED_PIN    15
8  #define TRAZA_PIN  14
9
10 #define ALARM_NUM   0
11 #define ALARM_IRQ   TIMER0_IRQ_0
12
13 #define INTERVALO_US 250000u // 250 ms
14
15 volatile uint32_t next_deadline;
16
17 // Rutina de atención a la interrupción
18 void on_alarm_irq(void)
19 {
20     // 1. Limpiar la interrupción de la alarma
21     hw_clear_bits(&timer_hw->intr, 1u << ALARM_NUM);
22
23     // 2. Cambiar el estado del LED
24     sio_hw->gpio_togl = 1u << LED_PIN;
25
26     // 3. Generar la traza para el osciloscopio
27     sio_hw->gpio_togl = 1u << TRAZA_PIN;
28
29     // 4. Calcular el siguiente instante de interrupción
30     next_deadline += INTERVALO_US;
31
32     // 5. Programar nuevamente la alarma
33     timer_hw->alarm[ALARM_NUM] = next_deadline;
34 }
35
36
37 int main(void)
38 {
39     // -----
40     // Configuración GPIO LED
41     // -----
42
43     gpio_init(LED_PIN);
44
45     // Configurar GP15 como salida usando SIO
46     sio_hw->gpio_oe_set = 1u << LED_PIN;
47
48     // LED inicialmente apagado
49     sio_hw->gpio_clr = 1u << LED_PIN;
50
51

```

```

52 // -----
53 // Configuración GPIO de traza
54 // -----
55
56 gpio_init(TRAZA_PIN);
57
58 // Configurar GP14 como salida usando SIO
59 sio_hw->gpio_oe_set = 1u << TRAZA_PIN;
60
61 // Traza inicialmente en LOW
62 sio_hw->gpio_clr = 1u << TRAZA_PIN;
63
64
65 // -----
66 // Configuración de la IRQ
67 // -----
68
69 // Registrar la función que atenderá la interrupción
70 irq_set_exclusive_handler(ALARM_IRQ, on_alarm_irq);
71
72 // Limpiar cualquier interrupción pendiente
73 hw_clear_bits(&timer_hw->intr, 1u << ALARM_NUM);
74
75 // Habilitar la alarma 0 dentro del TIMER
76 hw_set_bits(&timer_hw->inte, 1u << ALARM_NUM);
77
78 // Habilitar la IRQ en el NVIC
79 irq_set_enabled(ALARM_IRQ, true);
80
81
82 // -----
83 // Programar primera alarma
84 // -----
85
86 // Leer el contador actual del temporizador
87 uint32_t ahora = timer_hw->timerawl;
88
89 // Primera interrupción dentro de 250 ms
90 next_deadline = ahora + INTERVALO_US;
91
92 // Programar alarma
93 timer_hw->alarm[ALARM_NUM] = next_deadline;
94
95
96 // -----
97 // Programa principal
98 // -----
99
100 while (true)
101 {
102     // El CPU queda libre.
103     // El temporizador funciona mediante interrupciones.
104     tight_loop_contents();
105 }
106 }
107

```

Resultados

Tabla de resultados: Mediciones del Osciloscopio		
Parámetro	Valor esperado (código)	Valor medido (osciloscopio)
Periodo	250 ms	500.0 ms (250ms medio ciclo)
Jitter (Max-Min o Std)	≈0 (ideal)	≈835.7 ns

Surfer Project:

Periodo promedio:

$$T_1 = t_2 - t_1 = 250.053112ms - 0ms = 250.053112ms$$

$$T_2 = t_3 - t_2 = 500.052352ms - 250.053112ms = 249.999240ms$$

$$T_3 = t_4 - t_3 = 750.0526ms - 500.052352ms = 250.000248ms$$

$$T_4 = t_5 - t_4 = 1000.052848ms - 750.0526ms = 250.000248ms$$

$$T_5 = t_6 - t_5 = 1250.053096ms - 1000.052848ms = 250.000248ms$$

$$T_{promedio} = \frac{\sum_{i=1}^n T_i}{n} = \frac{\sum_{i=1}^5 T_i}{5} = 250.0106192ms$$

Jitter por Max – Min:

$$T_{min} = 249.999240ms \quad T_{max} = 250.053112ms$$

$$J_{pp} = T_{max} - T_{min} = 250.053112ms - 249.999240ms = 0.053872ms$$

Jitter por desviación estándar:

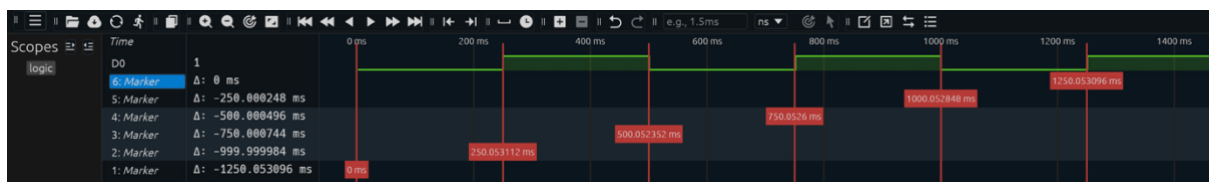
$$\sigma = \sqrt{\frac{\sum_{i=1}^n (T_i - T_{promedio})^2}{n}} \approx 0.021250ms$$

Evidencias

Simulación Wokwi SIO: <https://wokwi.com/projects/473743004155018241>

El siguiente enlace tiene una vigencia al 28 de Septiembre del 2026. Sin embargo, el archivo del video se puede encontrar dentro del zip adjuntado a la entrega de esta práctica.

[SE1_P3_EQ3_EVIDENCIA.mp4](#)



Conclusiones

En esta práctica se logró implementar en diferentes entornos de prueba, un código que permitió generar un temporizador de sistema con IRQ y trazas por toggling, y medir el jitter generado. Aunque se utilizó la misma configuración física (protoboard, LED, conexiones) y el mismo código en los diferentes entornos, se obtuvieron diferentes resultados que no implican la nulidad de alguno de los ellos.

Inicialmente, la práctica logró implementar un temporizador de sistema mediante interrupciones IRQ, siendo la función `void on_alarm_irq(void) { ... }` la ejecución estrella que se llevó a cabo mientras ocurre la IRQ resaltando la independencia del temporizador respecto al código principal. Además, de que dentro de dicha función el GPIO14 se utilizó como un pin de traza. Es decir, que realiza un toggle observable como un cambio de voltaje mediante un osciloscopio y mediante Surfer Project para la simulación en Wokwi; señal de cual se logró medir el jitter producido por pequeñas variaciones en el instante real en que se atiende y ejecuta cada interrupción.

El jitter medido en el osciloscopio fue de 0.0008357ms, mientras que el jitter medido en Surfer Project fue mayor con valor de **0.053872ms** el jitter por max-min; y de **0.021250ms** el jitter por desviación estándar. Fue necesario calcular mediante ecuaciones el jitter producido en la simulación obtenida por Wokwi, ya que Surfer Project únicamente puede agregar marcadores de tiempo.

Sin embargo, ambas medidas siguen considerándose tan pequeñas y podemos considerar dichas variaciones, para el propósito de esta práctica, como despreciables y decir que son aproximadamente igual a 0. Cumpliendo de esta forma la obtención de un tiempo de 250ms entre interrupciones que marcarán los cambios entre estados del led por toggling.

Declaración de IA

1. ChatGPT
 - a. Comentado de código
 - b. Obtención de las ecuaciones para calcular el periodo promedio, el jitter por max – min y el jitter por desviación estándar
 - c. Localización de fragmentos de código para ejemplificar en la conclusión
 - d. Corrección de sintaxis para la conclusión (mas no redacción de la misma)
2. Claude
 - a. Búsqueda de fuentes para el marco teórico
 - b. Comentado de código
3. Lucid AI
 - a. Generación del esquemático del código
4. Symbolab
 - a. Cálculo de las ecuaciones planteadas en el apartado de resultados

Referencias

Apache NuttX. (s. f.). Raspberry Pi RP2350.
<https://nuttx.apache.org/docs/latest/platforms/arm/rp23xx/index.html>

Llamas, L. (s. f.). Interrupciones IRQ en GPIO con MicroPython.
<https://www.luisllamas.es/micropython-interrupciones-irq-gpio/>

García, I. (2026). Qué es el jitter en una red y cómo afecta a tu conexión.
<https://pandorafms.com/blog/es/jitter-en-redes-it/>